

QRNG Service: Randomness-as-a-Service

Architecture and Implementation of a Cloud Quantum Random Number Generator

Manuel Ganino
Student at University of Calabria
`ganino.manuel01@gmail.com`

Contents

1	Introduction and Context	2
1.1	The Problem of Randomness in Computer Science	2
1.2	The Quantum Approach (QRNG)	2
1.3	Project Objectives	2
2	Theoretical-Mathematical Foundations	4
2.1	The Qubit and Superposition	4
2.2	Quantum Noise (Hardware Bias)	4
2.3	Post-Processing and the Von Neumann Extractor	4
2.4	Statistical Certification (NIST SP 800-22)	5
3	System Architecture	6
3.1	Infrastructure Overview	6
3.2	The In-Memory Entropy Pool	6
3.3	Asynchronous Processing	6
4	Engineering Challenges and Solutions	8
4.1	Concurrency Management (Race Conditions)	8
4.2	Task Synchronization (Distributed Locks)	8
4.3	Resilience and Graceful Degradation	8
4.4	Performance Optimization	9
5	Deployment and Operations	10
5.1	Containerization with Docker and Docker Compose	10
5.2	Security Management and Environment Variables	10
5.3	Reproducibility and Continuous Deployment (CI/CD)	11
6	Results Analysis	12
6.1	Extraction Efficiency	12
6.2	NIST Test Results	12
6.3	Latency and Response Times	12
7	Conclusions and Future Developments	13

Chapter 1

Introduction and Context

1.1 The Problem of Randomness in Computer Science

Random number generation is a fundamental pillar of modern computer science, with applications ranging from stochastic simulations (Monte Carlo methods) to the critical sector of cryptography. However, classical computers are intrinsically deterministic machines: given the same initial state and a set of instructions, they will always produce the exact same result.

To overcome this limitation, the software industry commonly relies on Pseudo-Random Number Generators (PRNG). These algorithms use a mathematical formula to produce sequences of numbers that appear random, starting from an initial value called a *seed*. The critical limit of PRNGs lies precisely in their determinism: if an attacker manages to deduce the initial seed (often derived from the system clock or other predictable variables), they can regenerate the entire cryptographic sequence, compromising RSA keys, TLS sessions, or authentication tokens.

For high-level cryptography, it is imperative to use True Random Number Generators (TRNG), which do not rely on mathematical formulas but measure entropy from unpredictable physical phenomena (such as the thermal noise of circuits).

1.2 The Quantum Approach (QRNG)

While classical mechanics is deterministic, quantum mechanics is the only domain in physics where unpredictability is a fundamental property demonstrable in nature.

A Quantum Random Number Generator (QRNG) exploits principles such as Heisenberg's uncertainty principle and wave function collapse. By placing a qubit in a state of perfect quantum superposition (for example, by applying a Hadamard logic gate), the outcome of its measurement is physically impossible to predict before the measurement itself occurs. The result will be a "1" or a "0" with exactly identical probabilities, providing the purest and safest form of entropy known to modern science.

1.3 Project Objectives

This project aims to bridge the gap between complex research on quantum hardware and everyday software development. Access to real quantum computers, such as those offered by IBM Quantum, is typically limited by architectural barriers, primarily the very high latency due to processing queues (which can take from several minutes to hours).

The result of this work is the design and implementation of a highly scalable cloud-native service offering Randomness-as-a-Service. The specific objectives achieved are:

- **Real Extraction:** Direct interfacing with physical quantum processors via the Qiskit library, avoiding the use of local classical simulators.
- **Latency Masking:** Development of an asynchronous microservices architecture (via Celery and RabbitMQ/Redis) capable of accumulating quantum entropy in the background, making it available to end-users with response times in the order of milliseconds.
- **Security and Certification:** Implementation of post-processing algorithms (Von Neumann Extractor) to eliminate hardware bias and subsequent certification of randomness through the rigorous NIST SP 800-22 statistical suite.
- **High Availability:** Implementation of distributed lock mechanisms, atomic transactions to prevent race conditions, and graceful degradation routines (fallback to cryptographically secure PRNGs in case of quantum pool depletion).

The resulting architecture demonstrates the technical feasibility of integrating quantum computing paradigms within modern, containerized RESTful Web infrastructures.

Chapter 2

Theoretical-Mathematical Foundations

2.1 The Qubit and Superposition

Unlike the classical bit, which can exclusively assume the value 0 or 1, the fundamental unit of quantum information is the qubit (quantum bit). A qubit can exist in a linear combination, or superposition, of the two basis states, typically denoted with Dirac notation as $|0\rangle$ and $|1\rangle$.

To extract entropy reliably, it is necessary to manipulate the qubit so that the probability of measuring the two states is identical. This result is obtained by applying the Hadamard Gate (H) to a qubit initialized to the state $|0\rangle$. The operation transforms the state of the qubit as follows:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

In this state of perfect superposition, the laws of quantum mechanics postulate that the probability of measuring $|0\rangle$ is exactly 50%, as is that of measuring $|1\rangle$. Until the moment of measurement, the collapsed value is unknowable, guaranteeing the generation of pure random data.

2.2 Quantum Noise (Hardware Bias)

Although the theoretical model guarantees a 50/50 probability, current physical quantum processors (often defined as NISQ - Noisy Intermediate-Scale Quantum devices) are subject to environmental and engineering imperfections.

Qubits tend to undergo decoherence and thermal relaxation phenomena (T_1 relaxation), losing energy and spontaneously decaying towards the ground state $|0\rangle$. Consequently, a measurement on real hardware will not return a perfectly balanced distribution but will present a bias (for example, a 52% probability of measuring 0 and 48% of measuring 1). In cryptography, any bias makes a sequence less secure, as it reduces the search space for a potential attacker.

2.3 Post-Processing and the Von Neumann Extractor

To mitigate the hardware bias problem and guarantee the cryptographic purity of the sequence, it is essential to apply a post-processing algorithm. In this project, the classic Von Neumann Extractor was implemented.

The algorithm analyzes the raw bit string by reading it in (non-overlapping) pairs and applies the following set of deterministic rules:

- If the pair is 01, the bit 0 is extracted.

- If the pair is 10, the bit 1 is extracted.
- If the pair is 00, it is discarded.
- If the pair is 11, it is discarded.

The mathematical assumption behind the algorithm is that, regardless of the source's bias (provided the measurements are independent), the probability of obtaining the sequence "01" is exactly equal to the probability of obtaining "10". Although this process discards approximately 75% of the original raw bits, the surviving bits are perfectly balanced and free from systematic bias.

2.4 Statistical Certification (NIST SP 800-22)

Theoretical validation is not sufficient for cryptographic use in production. To empirically certify the absence of recognizable patterns in the post-processed sequence, the system was subjected to the National Institute of Standards and Technology (NIST) tests.

The NIST SP 800-22 suite includes 15 complex statistical tests, including the Frequency Test (to confirm the overall proportion of zeros and ones) and the Runs Test (to analyze the length of consecutive sequences of identical bits). A test is considered passed if its probability value (P-value) exceeds a certain significance threshold (α , typically set to 0.01). The automation of these tests on the collected entropy pools guarantees that the service exclusively delivers data formally indistinguishable from true randomness.

Chapter 3

System Architecture

3.1 Infrastructure Overview

To transform quantum entropy extraction into a usable and scalable service, the system was engineered following the microservices architectural pattern. The infrastructure consists of four main modules, isolated through containerization, that communicate asynchronously:

- **Backend REST API (FastAPI):** The access point for the end user. It handles HTTP requests and extracts data in real-time.
- **In-Memory Database (Redis):** Acts as a reservoir ("pool") for pre-calculated entropy and as a message broker for work queues.
- **Asynchronous Worker (Celery):** The background process tasked with interfacing with IBM's quantum computers via the Qiskit library.
- **Orchestrator (Python):** A monitoring daemon that constantly checks the entropy pool levels and triggers refill tasks when necessary.

3.2 The In-Memory Entropy Pool

The fundamental problem with cloud-based quantum hardware is the unpredictable latency caused by IBM's execution queues. If the system required quantum entropy on-demand at the time of the user's HTTP call, the response times (Timeout) would render the service unusable for real-time applications.

The adopted solution involves decoupling generation from distribution. The system pre-calculates the entropy and stores it in an ultra-high-performance in-memory database: Redis. Redis allows atomic read and write operations with a time complexity of $\mathcal{O}(1)$. When a user makes a request to the API, the FastAPI service instantly retrieves the requested bits from Redis (with response times in the order of 2 - 5 milliseconds), guaranteeing a latency indistinguishable from that of a standard local PRNG.

3.3 Asynchronous Processing

The decoupling is made possible by the implementation of an asynchronous worker based on Celery. This component executes the computationally intensive work (quantum network communication) in a process entirely separate from the web HTTP request lifecycle.

The asynchronous flow unfolds as follows:

1. The Orchestrator detects that the Redis pool size has dropped below a specific security threshold (e.g., 50,000 bits).
2. The Orchestrator inserts a refill task into the Celery message queue.
3. The Worker picks up the task, authenticates with the IBM Quantum platform, and automatically selects the physical processor that is currently the least crowded.
4. The quantum circuit is generated and sent. At this point, the Worker listens asynchronously, waiting for the job results.
5. Once the raw bits are received from the IBM hardware, the Worker locally executes the Von Neumann Extractor.
6. The cryptographically pure entropy is finally appended to the Redis pool, making it immediately ready for consumption by the API.

Chapter 4

Engineering Challenges and Solutions

4.1 Concurrency Management (Race Conditions)

Exposing the entropy pool via a public REST API introduces the problem of concurrent access. If two or more users made a request in the exact same millisecond, the backend could read the same block of bits for both, effectively serving the same cryptographic seed to different entities and irreversibly compromising their security.

To solve this problem, retrieving bits from Redis was encapsulated in an atomic transaction leveraging the `WATCH` and `MULTI` commands. When the API reads the pool key, it places it under strict observation. If another thread modifies the pool before the current transaction concludes, Redis aborts the operation, raising a `WatchError` exception. The FastAPI backend intercepts the exception and automatically retries the extraction within a `while` loop, guaranteeing absolute mutual exclusion without locking the entire database.

4.2 Task Synchronization (Distributed Locks)

The intrinsically high latency of quantum hardware poses a significant challenge for the refill process. If the pool level fell below the critical threshold and the IBM job took hours to complete, the Orchestrator would relentlessly continue to send new tasks to Celery in a loop, clogging the queues and draining the cloud account's limit quota.

The engineered solution is a Distributed Lock also based on Redis. Before queuing a new quantum task, the Orchestrator attempts to acquire an exclusive key (`LOCK_KEY`) using the `SET NX` (Set If Not Exists) construct. To prevent infinite stalls (deadlocks) in the event of an anomalous worker crash, the key is equipped with an automatic safety expiration (Time-To-Live) set to 2 hours. The final release of the lock is guaranteed at the end of the processing by a robust `finally` block in the Celery worker code.

4.3 Resilience and Graceful Degradation

In anomalous traffic scenarios (e.g., traffic spikes linked to intensive use by a user), the quantum pool could deplete faster than the worker's refill capacity. Systematically responding with an HTTP 503 Service Unavailable error code would severely compromise the client service's reliability (uptime).

Therefore, the system was designed with a Graceful Degradation pattern. Should the Redis database report an insufficient entropy level, the backend activates a silent Fallback mechanism, resorting to Python's native `secrets` library (a cryptographically secure PRNG based on the operating system's entropy). The JSON response is then integrated with an optional `warning` pa-

parameter to transparently notify the calling developer of the temporary switch from the quantum source to the classical one.

4.4 Performance Optimization

In order to maximize the architecture's overall throughput and cut down clock cycles spent on the server processor, the most intensive data processing functions were optimized at a low level:

- **Von Neumann Extractor:** Traditional iterative loops for string checking were replaced by C-optimized regular expressions (`re.findall('..', raw_bits)`), drastically reducing Python execution times for post-processing tens of thousands of bits.
- **Binary Management:** The conversion of very long quantum binary strings into hexadecimal format (necessary for the JSON payload) was optimized by grouping the information into octets and allocating memory contiguously using native `bytearray` objects.

Chapter 5

Deployment and Operations

5.1 Containerization with Docker and Docker Compose

To ensure that the system is agnostic to the underlying infrastructure and to eliminate the problem of local dependencies (the classic “it works on my machine” anti-pattern), the entire microservices architecture was containerized using Docker.

The configuration relies on a lean `Dockerfile`, derived from the official `python:3.11-slim` image, which handles the installation of classical and quantum dependencies in a pre-configured and isolated environment. Container orchestration is managed declaratively via Docker Compose. The `docker-compose.yml` file defines four interconnected services:

- **redis:** The in-memory database for the entropy pool.
- **api:** The Uvicorn server exposing the FastAPI application on port 8000.
- **worker:** The Celery process managing integration with quantum hardware and post-processing.
- **orchestrator:** The Python daemon constantly monitoring pool levels.

Docker Compose automatically creates a private virtual network (bridge network) between these containers, allowing them to communicate via hostname (e.g., `REDIS_HOST=redis`) without exposing internal ports outside the host system.

5.2 Security Management and Environment Variables

The source code of a cloud-native application must never contain hard-coded credentials. In the context of this project, the access token for the IBM Quantum infrastructure represents a critical secret: if compromised, it could allow third parties to exhaust processing quotas or sabotage quantum provisioning.

To mitigate this risk, security was implemented following the strict principles of the Twelve-Factor App. Credentials and network configurations were injected into the containers at run-time via Environment Variables. The IBM token is read from a local `.env` file (strictly excluded from Git versioning via `.gitignore`) and passed securely and exclusively to the Worker container via interpolation. In this way, the public API web server does not even have access to the quantum key, applying the Principle of Least Privilege and drastically reducing the attack surface.

5.3 Reproducibility and Continuous Deployment (CI/CD)

The Infrastructure-as-Code (IaC) approach adopted for deployment makes the Cloud QRNG service immediately reproducible on any machine, server, or cloud cluster supporting the Docker engine.

To initialize the entire system (from downloading base images to building containers, starting the database, and queuing the first tasks), the operator simply needs to run a single command: `docker-compose up -d --build`. This extreme standardization lays solid foundations for the implementation of CI/CD (Continuous Integration / Continuous Deployment) pipelines. The architecture is indeed ready to be integrated with tools like GitHub Actions, capable of running automated tests and statistical validation at every new Pull Request, and then deploying updates to production without downtime (Zero-Downtime Deployment).

Chapter 6

Results Analysis

6.1 Extraction Efficiency

The implementation of the Von Neumann Extractor confirmed theoretical expectations regarding the trade-off between the quantity and quality of entropy. During execution tests, against a raw payload of 5,000 bits returned by IBM processors (e.g., `ibm_fez`), the algorithm discarded the pairs correlated by thermal noise, returning on average 1,200-1,250 purified bits. This physiological loss of $\sim 75\%$ is a justified computational cost, as it transforms a “noisy” quantum source (NISQ) into a bitstream with perfect balance (50% probability for states 0 and 1), fundamental for the reliability of cryptographic seeds.

6.2 NIST Test Results

The purified entropy was accumulated and subjected to the entire battery of tests by the National Institute of Standards and Technology (NIST SP 800-22). All applicable tests, including Frequency, Runs, and Cumulative Sums, registered a P-value well above the critical threshold ($\alpha = 0.01$). This mathematically certifies that our REST API’s output is cryptographically secure and free of predictable patterns.

6.3 Latency and Response Times

From an architectural perspective, the combination of Celery and Redis successfully resolved the quantum hardware bottleneck. The intrinsic latencies of cloud interaction with physical QPUs (Quantum Processing Units), which vary from 15 seconds up to several hours in case of queue congestion, were completely masked from the end user. GET requests to the FastAPI endpoint are resolved by fetching the pre-calculated bits from RAM memory, guaranteeing response times below 5 milliseconds.

Chapter 7

Conclusions and Future Developments

This project tangibly demonstrates the feasibility of integrating quantum computing processes into cloud-native software architectures. By leveraging an asynchronous microservices pattern and Docker containerization, a robust, fault-tolerant, and production-ready Randomness-as-a-Service was achieved.

Future developments to further scale the infrastructure include:

- **API Security:** Implementation of an Authentication system (API Keys) via JWT tokens and Rate Limiting policies to commercialize the service from a SaaS (Software-as-a-Service) perspective.
- **High Availability:** Migration of the infrastructure from Docker Compose to Kubernetes (K8s) to dynamically orchestrate FastAPI backend pod replicas and configure a Highly Available Redis cluster.
- **Automated Testing:** Writing comprehensive Unit Tests (e.g., via the `pytest` library) to automatically validate Distributed Locking mechanisms and Graceful Degradation activation in CI/CD.